Programming with C++ – Fundamentals

LSEG Technology

5th February 2026

Anupa Satharasinghe

LSEG

What you'll need today

- Option 1 (Recommended – Windows):

  Visual Studio Community Edition https://visualstudio.microsoft.com/vs/community/

- Option 2 (Any OS):

  Visual Studio Code with any C++ compiler

- Option 3 (No setup – for practice):

  Online compiler like Programiz Online Compiler

(Scan QR code for Programiz)

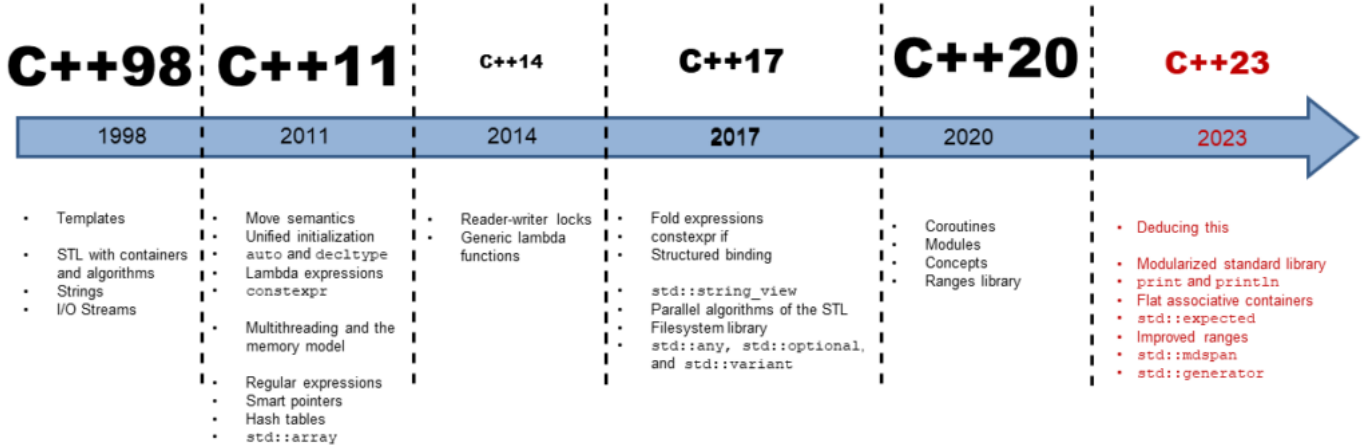
Agenda

- History of C++
- Why C++
- Hello World in C++
- Variables, Data Types and Operators
- Control Structures
- Functions
- Pointers
- Stages of Compilation
- Optimization Levels

History of C++

- Early 1980s – C with Classes (Bjarne Stroustrup)
- 1985 – C++ officially named
- 1998 – First ISO standard (C++98)
- 2011 – Modern C++ begins (C++11)
- Today – Actively evolving (C++14, C++17, C++20, C++23)

Key idea: C++ evolved to balance performance, control, and abstraction



LSEG

Why C++

- High performance with predictable behaviour
- Fine-grained control over memory and resources
- Scales well for large, complex systems
- Strong ecosystem and long-term stability

Widely used in:

- Game engines
- Financial & low-latency systems
- Embedded & real-time systems
- High-performance infrastructure

Hello World!

main.cpp

lecure > main.cpp

```cpp
#include <iostream> // Preprocessor directive: Tells the compiler to include the standard input/output library
// This is required for using std::cout

int main() {  // The main function is the entry point of every C++ program
    // Program execution starts here

    std::cout << "Hello, World!\n";  // std::cout is used to print output to the console
    // The << operator sends data to the output stream

    return 0;  // Ends the program - Returning 0 indicates successful execution
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[ec2-user@ip-10-230-3-10 lecure]$ g++ -o main main.cpp && ./main
Hello, World!
[ec2-user@ip-10-230-3-10 lecure]$
```

Variables

- Variables are names to pieces of memory

variables.cpp

lecure > variables.cpp

```cpp
#include <iostream>

int main() {
    int a;          // integer variable declared (no value yet)
    a = 42;         // value 42 assigned later

    int b = 42;     // integer variable declared and assigned in one step

    std::cout << "a = " << a << '\n';
    std::cout << "b = " << b << '\n';

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[ec2-user@ip-10-230-3-10 lecure]$ g++ -o variables variables.cpp && ./variables
a = 42
b = 42
```

Data Types

Tells what type of and size of data will be stored in a variable

Category	Type(s)	Meaning	Example Value	Notes
Boolean	bool	true or false	true	Used for conditions and decisions
Character	char	a single character	'A'	Always 1 byte; stores characters
Integer	int	whole numbers	42	Size is implementation-defined (usually 32-bit today)
Floating-point	float	decimal number	3.14	Less precision than double
Floating-point	double	decimal number	3.141592653	Preferred for most decimal calculations
Fixed-width integer	std::int32_t	exactly 32-bit integer	1000	Use when size must be guaranteed
No value	void	no data	—	Used for functions that return nothing

Data Types

datatypes.cpp

lecure > datatypes.cpp

```cpp
#include <iostream>

int main() {

    // Declare and initialize variables
    int    year        = 2026;        // integer (whole number)
    float  temperature = 36.5;        // decimal, when lower precision is fine
    double pi          = 3.141592653; // decimal, when higher precision needed
    char   grade       = 'A';         // single character
    bool   isCppFun    = true;        // boolean

    // Print variables with labels
    std::cout << "year            = " << year << "\n";
    std::cout << "temperature     = " << temperature << "\n";
    std::cout << "pi              = " << pi << "\n"; //by default std::cout will only print 6 significant digits
    std::cout << "grade           = " << grade << "\n";
    std::cout << "isCppFun        = " << isCppFun << "\n";

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - lecure

```
[ec2-user@ip-10-230-3-10 lecure]$ g++ -o datatypes datatypes.cpp && ./datatypes
year            = 2026
temperature     = 36.5
pi              = 3.14159
grade           = A
isCppFun        = 1
```

Data Types

datatypes.cpp

lecure > datatypes.cpp

```cpp
#include <iostream>
#include <iomanip> // allows us to use std::setprecision

int main() {

    // Declare and initialize variables
    int    year        = 2026;        // integer (whole number)
    float  temperature = 36.5;        // decimal, when lower precision is fine
    double pi          = 3.141592653; // decimal, when higher precision needed
    char   grade       = 'A';         // single character
    bool   isCppFun    = true;        // boolean

    // Print variables with labels
    std::cout << "year            = " << year << "\n";
    std::cout << "temperature     = " << temperature << "\n";
    std::cout << "pi              = " << std::setprecision(10) << pi << "\n";
    //by default std::cout will only print 6 significant digits. Using setprecison we can change that (need to include <iomanip>)
    std::cout << "grade           = " << grade << "\n";
    std::cout << "isCppFun        = " << isCppFun << "\n";

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - lecure

```
[ec2-user@ip-10-230-3-10 lecure]$ g++ -o datatypes datatypes.cpp && ./datatypes
year            = 2026
temperature     = 36.5
pi              = 3.141592653
grade           = A
isCppFun        = 1
```

LSEG

Preprocessor Directives

Preprocessor directives are instructions handled before compilation.

Commonly used directives:

- #include – include header files
- #define – define constants or macros
- #if / #ifdef / #ifndef – conditional compilation
- #error – generate a compile-time error
- #pragma – compiler-specific instructions

Preprocessor Directives

multiplicationTable.cpp | datatypes.cpp

lecure > datatypes.c ~/lecure/multiplicationTable.cpp

```cpp
#include <iostream>
#include <iomanip> // allows us to use std::setprecision

#define NEXT_YEAR    2027 //precprocessor directive to define a constant

int main() {

    // Declare and initialize variables
    int    year        = 2026;        // integer (whole number)
    float  temperature = 36.5;        // decimal, when lower precision is fine
    double pi          = 3.141592653; // decimal, when higher precision needed
    char   grade       = 'A';         // single character
    bool   isCppFun    = true;        // boolean

    // Print variables with labels
    std::cout << "year            = " << year << "\n";
    std::cout << "temperature     = " << temperature << "\n";
    std::cout << "pi              = " << std::setprecision(10) << pi << "\n";
    //by default std::cout will only print 6 significant digits. Using setprecison we can change that (need to include <iomanip>)
    std::cout << "grade           = " << grade << "\n";
    std::cout << "isCppFun        = " << isCppFun << "\n";
    std::cout << "Next year       = " << NEXT_YEAR << "\n"; //printing a constant defined using preprocessor directive

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS bash - lecure

```
[ec2-user@ip-10-230-3-10 lecure]$ g++ -o datatypes datatypes.cpp && ./datatypes
year            = 2026
temperature     = 36.5
pi              = 3.141592653
grade           = A
isCppFun        = 1
Next year       = 2027
```

#define performs text substitution before compilation.

Here we use it to define a constant

Arrays

- An array is a collection of multiple values of the same type
- All elements are stored next to each other in memory
- Each element is accessed using an index
- Indexes start at 0 in C++
- Array size is fixed once created
- Accessing an invalid index leads to undefined behaviour

Arrays

arrays.cpp

~/lecure/arrays.cpp

```cpp
#include <iostream>

int main() {

    // 1) Integer array
    int scores[4] = {85, 90, 78, 81};

    std::cout << "scores[0] = " << scores[0] << "\n"; //access the first element
    std::cout << "scores[3] = " << scores[3] << "\n"; //access the 4th element

    // 2) Double array
    double temperatures[4] = {36.5, 37.0, 36.8, 36.6};

    std::cout << "temperatures[0] = " << temperatures[0] << "\n"; //access the first element
    std::cout << "temperatures[2] = " << temperatures[2] << "\n"; //access the 3rd element

    // 3) Character array
    char grades[4] = {'A', 'B', 'C', 'D'};

    std::cout << "grades[0] = " << grades[0] << "\n"; //access the first element
    std::cout << "grades[1] = " << grades[1] << "\n"; //access the 2nd element

    // 4) Boolean array
    bool attendance[4] = {true, false, true, true};

    std::cout << "attendance[0] = " << attendance[0] << "\n"; //access the first element
    std::cout << "attendance[3] = " << attendance[3] << "\n"; //access the 4th element

    // std::cout << "attendance[4] = " << attendance[4] << "\n"; Out of bounds (there is no 5th element)
    // This will lead to undefined behaviour

    return 0;
}
```

LSEG

Operators

Category	Operator(s)	Example	Notes
Assignment	=	x = 5; x = y = 10;	
Arithmetic	+, -, *, /, %	x = 11 % 3	x gets value ‘2’
Compound assignment	+=, -=, *=, /=	x += 5;	Same as x = x + 5;
Increment and decrement	++, --	x++ --x	Has suffix and prefix forms
Comparison	==, !=, <, >, <=, >=		
Logical	!, &&, ||	if ((a > 5) && (b < 10))	
Bitwise	&, |, ^, ~, <<, >>		

Operator precedence

Use parentheses when you need to override

Operators

Operator precedence decides which operator is evaluated first

- runs before + and -
- Comparisons (> < ==) run after arithmetic
- Logical && runs before logical ||
- Parentheses () override precedence
- If unsure, use parentheses for clarity

Associativity decides the direction of evaluation

- Applies when operators have the same precedence
- Most arithmetic operators associate left to right
- Assignment operators associate right to left

Control Flow

- Control flow decides which code runs and when
- Without control flow, programs run top to bottom only
- Control flow lets programs:
    - Make decisions
    - Repeat work
    - React to conditions

Three core concepts:

- Sequence
- Selection
- Repetition

Control Flow

- Main selection control flow forms:
  - If
  - if … else
  - if … else if … else
  - switch

```cpp
int number = 3;

if (number > 0) {
    std::cout << "Number is positive" << std::endl;
}
else if (number == 0) {
    std::cout << "Number is zero" << std::endl;
}
else {
    std::cout << "Number is negative" << std::endl;
}
```

```cpp
    // Day of the week
// 1 = Monday, 2 = Tuesday, ..., 7 = Sunday
int day = 6;   // try values from 1 to 7

switch (day) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
        std::cout << "Weekday" << std::endl;
        break;

    case 6:
    case 7:
        std::cout << "Weekend" << std::endl;
        break;

    default:
        std::cout << "Invalid day" << std::endl;
        break;
}
```

Control Flow

- Main repetition control flow forms:
  - While
  - do … while
  - For
  - break / continue (loop control)

```cpp
// Divide the numerator by each divisor in the array - skip 0 and stop when -1
int divisors[10] = {2, 0, 5, 0, 4, -1, 3, 1, 2, 0};
const int size = 10;

int numerator = 20;

std::cout << "=== Using for loop ===\n";
for (int i = 0; i < size; ++i) {

    if (divisors[i] == 0) {
        // Skip dividing by zero
        std::cout << "Skip: divisor is 0\n";
        continue;
    }

    if (divisors[i] == -1) {
        // Stop further processing
        std::cout << "Stop: sentinel -1 encountered\n";
        break;
    }

    std::cout << numerator << " / " << divisors[i]
              << " = " << (numerator / divisors[i]) << '\n';
}
```

```cpp
std::cout << "\n=== Using while loop ===\n";
int j = 0;
while (j < size) {

    if (divisors[j] == 0) {
        std::cout << "Skip: divisor is 0\n";
        ++j;                // advance before continue in while-loop
        continue;
    }
    if (divisors[j] == -1) {
        std::cout << "Stop: sentinel -1 encountered\n";
        break;
    }
    std::cout << numerator << " / " << divisors[j]
              << " = " << (numerator / divisors[j]) << '\n';

    ++j;
}
```

Exercise

- Write a program to generate the multiplication tables up to 12 x 12.

Exercise

multiplicationTable.cpp

lecure > multiplicationTable.cpp

```cpp
#include <iostream>

// Write a program to generate the multiplication tables up to 12 x 12

int main() {

    // Loop through numbers 1 to 12 (tables)
    for (int i = 1; i <= 12; i++) {

        std::cout << "Multiplication table for " << i << ":\n";

        // Loop through numbers 1 to 12 (multipliers)
        for (int j = 1; j <= 12; j++) {
            std::cout << i << " x " << j << " = " << (i * j) << "\n";
        }

        // Print a blank line after each table
        std::cout << "\n";
    }

    return 0;
}

// g++ -o multiplicationTable multiplicationTable.cpp && ./multiplicationTable

// Result looks like this :
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
[ec2-user@ip-10-230-3-10 lecure]$ g++ -o multiplicationTable multiplicationTable.cpp && ./multiplicationTable
Multiplication table for 1:
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5
1 x 6 = 6
1 x 7 = 7
```

Functions

- Function signature
- Function parameters and return values
- Function declaration vs. definition
- Pass by value, pass by reference

Functions

```
return_type name ( parameter1, parameter2 )
{
...
}
```

"body"

"parameter list"

- Function signature: The function's name and parameter types (and order). Used by the compiler to identify and overload functions.
- Function prototype: A declaration that tells the compiler the function exists, including the return type, name, and parameter types (ends with ;).

Functions

```cpp
#include <iostream>

// Function declaration (prototype)
int add(int a, int b);

int main() {
    int result = add(10, 20);
    std::cout << "Result = " << result << std::endl;
    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}
```

Functions

functionsParameterPassing.cpp > ...

```cpp
#include <iostream>

// ------------------------------
// Pass by VALUE
void incrementByValue(int x) {
    x = x + 1;   // modifies only the local copy
}

// ------------------------------
// Pass by REFERENCE
void incrementByReference(int& x) {
    x = x + 1;   // modifies the original variable
}

int main() {
    int number = 10;

    std::cout << "Initial value: " << number << std::endl;
    // Output: Initial value: 10

    incrementByValue(number);
    std::cout << "After pass by value: " << number << std::endl;
    // Output: After pass by value: 10

    incrementByReference(number);
    std::cout << "After pass by reference: " << number << std::endl;
    // Output: After pass by reference: 11

    return 0;
}
```

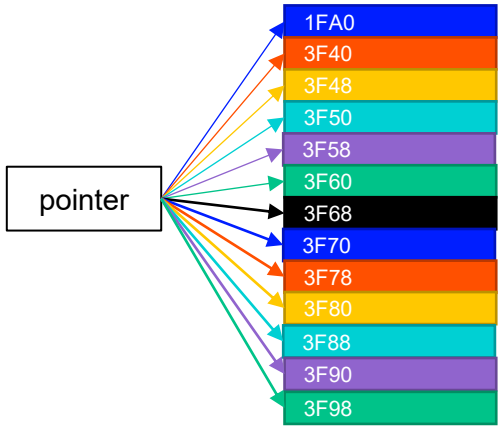
Exercise

Safe Factorial Function (refer functionsExercise.cpp for boilerplate)

Your task is to create a function called `factorial` with the following rules:

1. The function must calculate the factorial of a number.
2. Factorial is defined only for non-negative integers.
   - 0! = 1
3. If the input number is negative, the function must fail.
4. If the input number is greater than 12, the function must fail
   (to avoid integer overflow when using int).
5. The function must:
   - Return the factorial result
   - Use a reference parameter to indicate success or failure
6. Do NOT print anything inside the function.
7. Use a loop (do NOT use recursion).

C++ Pointers



- A pointer is just another variable, but it stores a memory address instead of a value.
- Why pointers exist
    - Dynamic memory
    - Large data without copying
    - Polymorphism (runtime behaviour)
    - Low-level / system programming
- Syntax

```
DataType*  pointerName = AddressOfVariable;
```

C++ Pointers

Address-of operator: &

- Get the address of a variable
- Example: &i gives the address of i

```
int i = 42;
int* pInt       = &i; //now pInt stores the address of i

cout << "pInt = " << pInt << endl;   //prints address of
cout << "*pInt = " << *pInt << endl; //dereferencing pInt with * gives 42
```

Dereference operator: *

- Access the value at the address stored in the pointer
- Example: *pInt gives the value of i

C++ Pointers

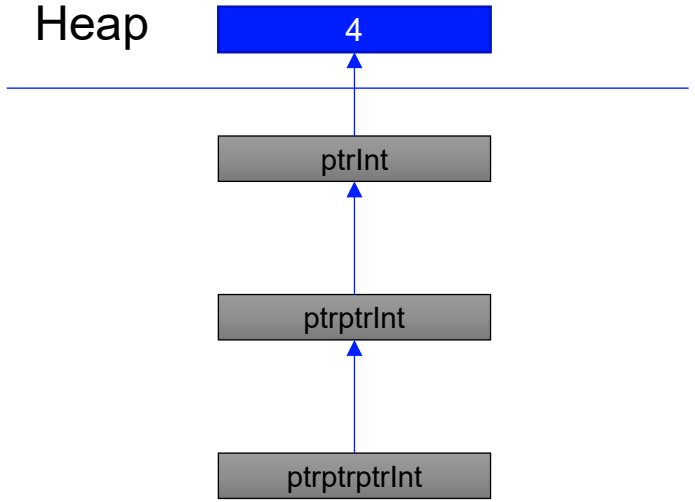
- What is stack
  - Store local variables
  - Store function parameters
  - Store container meta data
  - Close to the CPU → fast access
  - Limited resource
  - Automatic deallocation
- What is heap
  - Store dynamic variables
  - Slower than stack
  - Manual de-allocation
  - New allocates memory
  - Delete frees that memory

```cpp
// STACK MEMORY (automatic)

int stackInt = 10;                  // single int on the stack
int stackArray[3] = {1, 2, 3};       // array on the stack

std::cout << stackInt << '\n';
std::cout << stackArray[0] << '\n';


// HEAP MEMORY (dynamic)

// Single int on the heap
int* heapInt = new int(20);          // allocate int on heap
std::cout << *heapInt << '\n';
delete heapInt;                      // free heap memory


// Array on the heap
int* heapArray = new int[3]{4, 5, 6}; // allocate array on heap
std::cout << heapArray[0] << '\n';
delete[] heapArray;                   // free heap array
```

C++ pointers cntd..

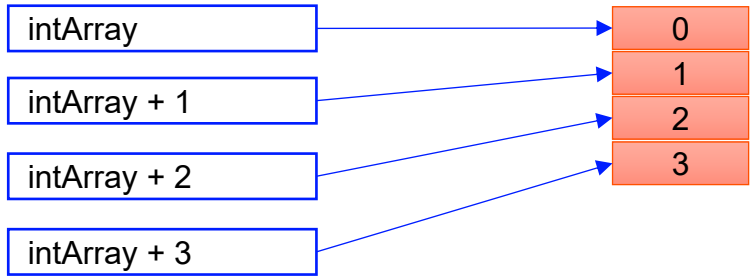
- Pointer to pointer

```
int main()
{
    int* ptrInt = new int(4);
    int** ptrptrInt = &ptrInt;
    int*** ptrptrptrInt = &ptrptrInt;

    delete ptrInt;
```



- Arrays as Pointers

```
int main()
{
    int intArray[4] = {0, 1, 2, 3};

    for (int i = 0; i < 4; ++i)
        std::cout << *(intArray + i) << " " << (intArray + i) << '\n';
```



Pointer arithmetic

```
int intArray[4] = {0, 1, 2, 3};

std::cout << "intArray        : " << intArray << '\n';
std::cout << "*intArray       : " << *intArray << '\n';

std::cout << "intArray + 1    : " << intArray + 1 << '\n';
std::cout << "*(intArray + 1) : " << *(intArray + 1) << '\n';

std::cout << "intArray + 2    : " << intArray + 2 << '\n';
std::cout << "*(intArray + 2) : " << *(intArray + 2) << '\n';

std::cout << "intArray + 3    : " << intArray + 3 << '\n';
std::cout << "*(intArray + 3) : " << *(intArray + 3) << '\n';
```

```
intArray        : 0x7ffc92b7e740
*intArray       : 0
intArray + 1    : 0x7ffc92b7e744
*(intArray + 1) : 1
intArray + 2    : 0x7ffc92b7e748
*(intArray + 2) : 2
intArray + 3    : 0x7ffc92b7e74c
*(intArray + 3) : 3
```

Function pointers

- Variables pointing to address of functions
- Syntax
  return type (* Name) (arg_types….)
- Usage
  Function indirection
  Virtual tables

```
int add(int a, int b)
{
    return a + b;
}

int multiply(int a, int b)
{
    return a * b;
}

int main()
{
    int (*operation)(int, int);

    operation = add;
    std::cout << operation(2, 3) << '\n'; // result is added: 5

    operation = multiply;
    std::cout << operation(2, 3) << '\n'; //result is multiplied: 6
}
```

LSEG

C++ references

- A pointer is an object that stores an address.
- A reference is an alias (another name) for an existing object.

- An alias to an existing object
- Must initialized at creation
- Cannot change the object after creation
- Never NULL

```cpp
int myInt = 1;
int& refMyInt = myInt;
std::cout << "myInt = 1" << std::endl << std::endl;
std::cout << "myInt           : " << myInt << std::endl;
std::cout << "refMyInt        : " << refMyInt << std::endl << std::endl;
myInt = 2;
std::cout << "myInt = 2" << std::endl << std::endl;
std::cout << "myInt           : " << myInt << std::endl;
std::cout << "refMyInt        : " << refMyInt << std::endl << std::endl;
refMyInt = 3;
std::cout << "refMyInt = 3" << std::endl << std::endl;
std::cout << "myInt           : " << myInt << std::endl;
std::cout << "refMyInt        : " << refMyInt << std::endl;
```

```
myInt = 1

myInt           : 1
refMyInt        : 1

myInt = 2

myInt           : 2
refMyInt        : 2

refMyInt = 3

myInt           : 3
refMyInt        : 3
```

Stages of Compilation

Pre-processor

- “g++ -E <input file> -o <output file>” will stop after pre-process stage and store the result in <output file>
- Pre-processor directives start with ‘#’
- e.g. #include replaces that line with the contents of the “included” file

Compiler

- “g++ -S <input file> -o <output file>” will stop after compilation stage and store the result in <output file>
- Compiles the “pre-processed” file and generates “assembly code”

Assembler

- “g++ -c <input file> -o <output file>” will stop after assembler. At this point, <output file> contains the “object code”. These files are called “object files”
- Converts the assembly code in to “object code”

Linker

- Combines “object files” with necessary system and user libraries to create “executables”

Optimization Levels

- Different optimization levels (GCC/Clang)
  - -O0 : No optimizations. Default. All debug information present. Less compilation time
  - -O1 :A set of optimizations applied. Take more time to compile.
  - -O2 : Recommended. Used in linux kernel too. More optimized. Nearly all supported optimizations that do not involve a space-speed tradeoff
  - -O3 : Not much recommended
  - -Os : Optimize for space
  - -Ofast : Optimize for speed. All O3 optimizations + some subtle differences. Not recommended
- Higher optimization → Higher Compiler Time → Faster Code, Less Debug information
- Lesser optimization level → Lesser Compiler Timer → Enough Debug information

- https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

References

- What every programmer should know about memory – Ulrich Drepper
- What every programmer should know about floating point – David Goldberg
- CppCon
- LearnCpp.com
- CPlusPlus.com Tutorial
- Tutorialspoint.com
- The Cherno

Thank You

anupa.satharasinghe@lseg.com